

Tecnologia de Comunicação

Módulo 2 – E1, códigos de linha, sincronização,
enquadramento e sinalização

Da telefonia digital à implementação de uma interface E1

Prof. Paulo Matias

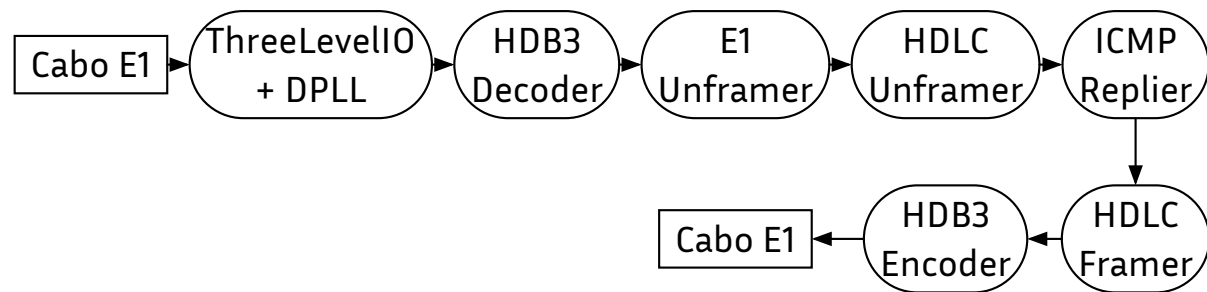
Objetivos deste módulo

Ao fim desta aula, o aluno deve saber responder:

- Por que o E1 usa HDB3 e não NRZ, AMI puro ou Manchester?
- Como o receptor recupera o clock e por que a DPLL converge?
- Como um fluxo serial vira timeslots E1 e depois quadros HDLC?
- Como o CRC detecta erros e por que ele cabe tão bem em hardware?
- Onde ficam voz, sinalização e dados dentro do quadro E1?

A prática não é uma coleção de blocos arbitrários — cada bloco resolve um problema físico concreto do enlace.

Pipeline da Prática 2: visão de sistema



DPLL: quando amostrar?

HDLCUnframer: onde começa/
termina o pacote?

HDB3Dec: que bits?

ICMPReplier: como gerar a
resposta?

E1Unframer: qual timeslot?

Voz, sinalização e dados: tudo no mesmo E1

- Historicamente, o E1 nasceu para voz PCM.
- Na Prática 2, nós o usamos para dados sobre HDLC.
- De modo geral, o mesmo E1 pode carregar:
 - voz codificada em PCM
 - sinalização CAS em TS16
 - dados síncronos em canais fracionados

Isso faz do E1 um excelente exemplo de integração entre teoria de telecomunicações e arquitetura de redes.

Notação do módulo 1 (recapitulação rápida)

No módulo 1, modelamos transmissão em banda base como:

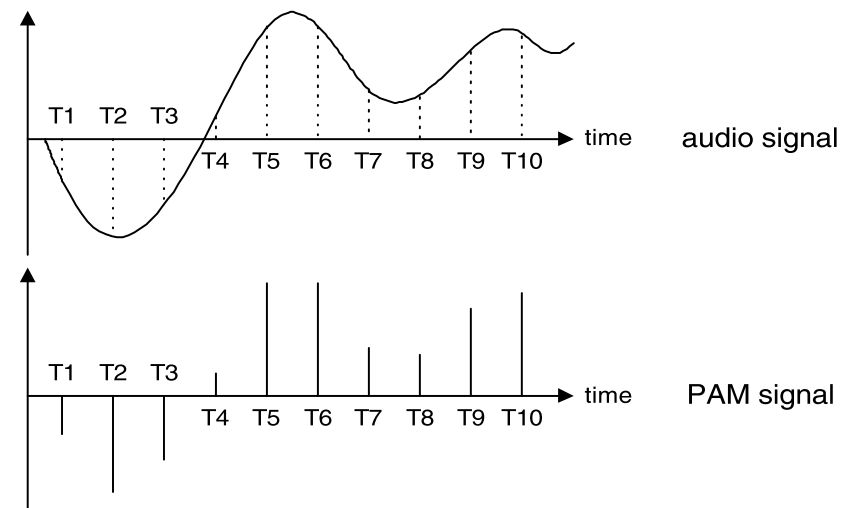
$$x(t) = \sum_n a[n] p(t - nT)$$

A mesma ideia continua valendo, mas agora o foco passa a ser:

- Quais formas de onda escolhemos para o meio físico
- Como detectar símbolos na presença de ruído, distorção e erro de temporização
- Como transformar a sequência de símbolos em estrutura de enlace

De voz analógica a 64 kbit/s

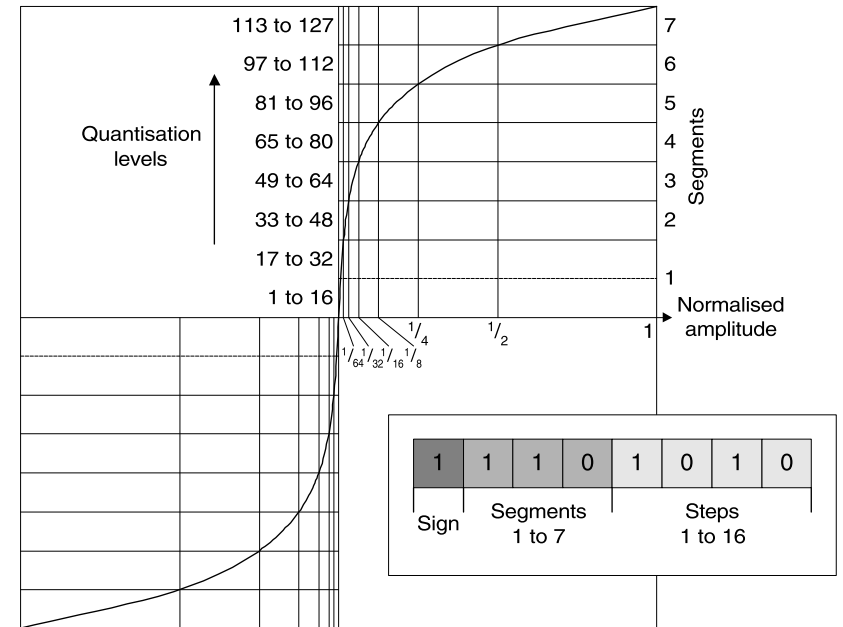
- Banda de voz telefônica:
300 Hz a 3400 Hz
- Amostragem a 8 kHz $\rightarrow$
1 amostra a cada 125 μ s
- Quantização em 8 bits $\rightarrow$
 $8000 \times 8 = 64$ kbit/s por canal
- Esse canal de 64 kbit/s é o
“átomo” do E1



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

Lei A e lei μ : por que companding?

- Quantização linear com poucos bits: sinais fracos sofrem muito mais ruído relativo.
- *Companding* redistribui os níveis de quantização.
- E1 / Europa / Brasil: **lei A**.
- T1 / EUA / Japão: **lei μ** .
- Intuição: mais resolução perto de zero, menos para amplitudes grandes.



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

Por que 2,048 Mbit/s?

- 1 quadro PCM dura **125 μ s**.
- Cada quadro tem **32 timeslots**.
- Cada timeslot tem **8 bits**.

$$32 \times 8 = 256 \text{ bits/quadro}$$

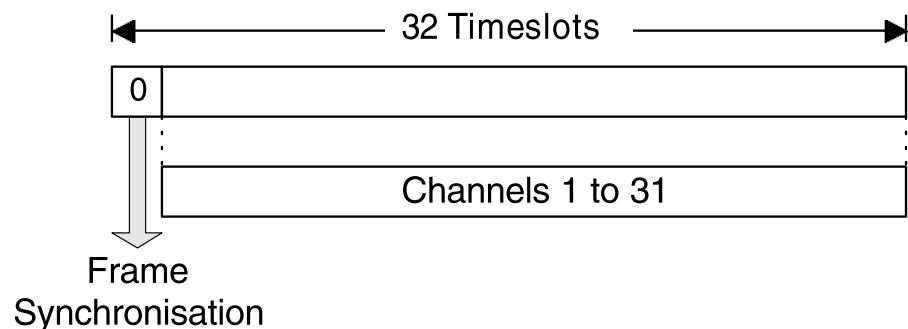
$$256 \frac{\text{bits}}{125 \mu s} = 2.048.000 \text{ bit/s}$$

- No PCM30: 30 canais de tráfego + TS0 (sincronismo) + TS16 (sinalização)

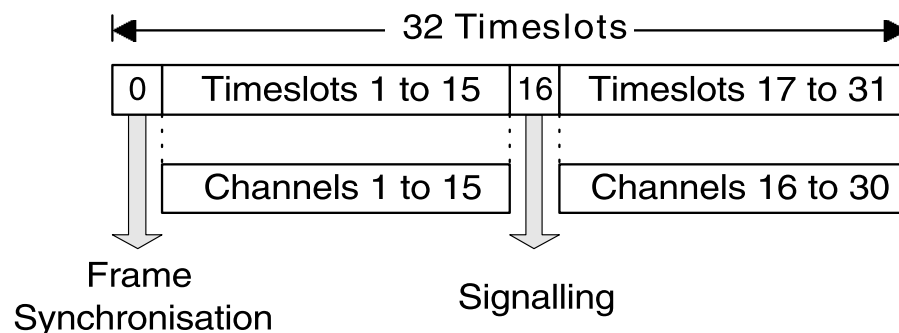
Esses sistemas nasceram antes dos microprocessadores modernos: os primeiros bancos PCM já faziam TDM e regeneração com eletrônica discreta.

Estrutura do quadro E1 segundo ITU-T G.704

PCM31

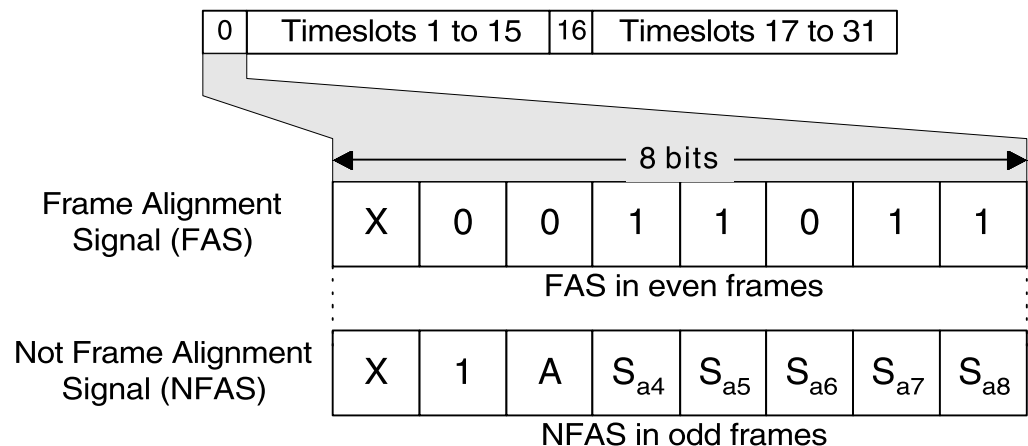


PCM30



- PCM31: TS0 para sincronismo, TS1–31 para tráfego.
- PCM30: TS0 para sincronismo, TS16 para sinalização, demais para tráfego.
- Na prática brasileira, o formato principal é o **PCM30**.

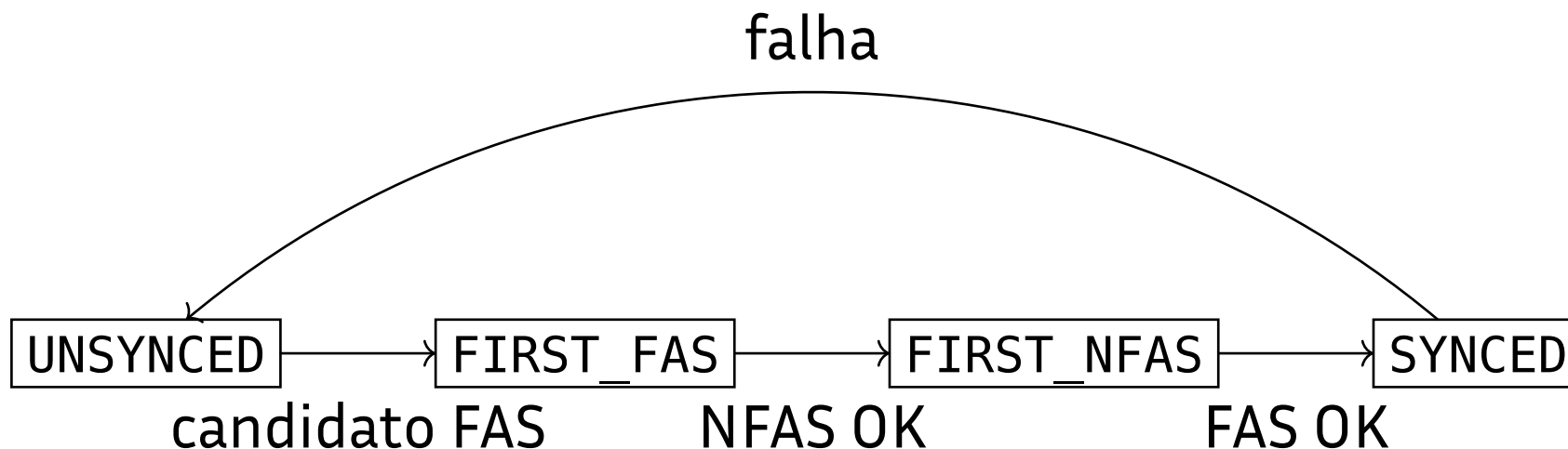
TS0: alternância FAS / NFAS



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

- Quadros pares: TS0 carrega o **FAS** (padrão 0011011 nos bits 2–8).
- Quadros ímpares: TS0 carrega o **NFAS**.
- Essa alternância permite ao receptor encontrar a fronteira de quadro.
- O bit 1 do TS0 não faz parte do padrão e pode ser reaproveitado para CRC-4.

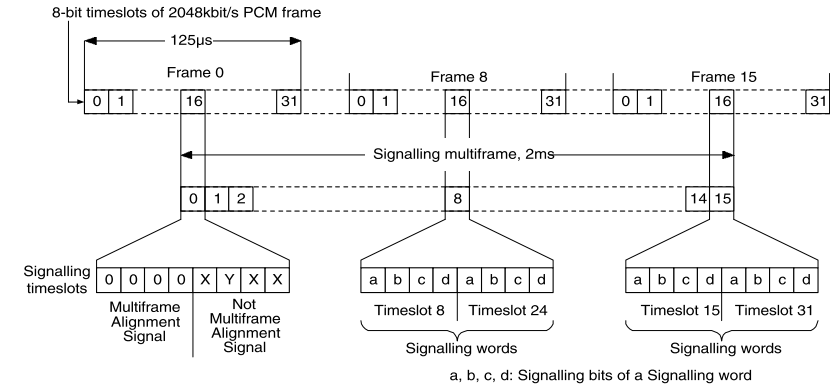
Máquina de estados do E1Unframer



Não basta encontrar “0011011” uma vez — é preciso provar que aquilo realmente é TSO e não coincidência nos dados.

TS16 e a multitrama de sinalização

- TS16 carrega sinalização CAS, não áudio.
- Uma multitrama = **16 quadros** = **2 ms**.
- Em cada TS16 cabem dois canais de sinalização ABCD.
- Taxa suficiente para representar sinais lentos como pulso decádico (dezenas de ms por pulso).



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

Sinalização é lenta; voz e dados são rápidos. Por isso faz sentido tirar a sinalização do canal de voz e colocá-la em TS16.

Alarmes em E1

- **RAI** (Remote Alarm Indication): bit A do NFAS avisa a ponta remota que algo está errado.
- **AIS** (Alarm Indication Signal): sequência quase toda em 1, para manter clock nos regeneradores.
- **Perda de frame sync**: três FAS incorretos consecutivos.
- **Perda de multiframe sync**: problema na estrutura de TS16.

A Prática 2 trata apenas o sincronismo por FAS/NFAS; os demais alarmes aparecem aqui como parte da tecnologia E1 real.

CRC-4 no E1

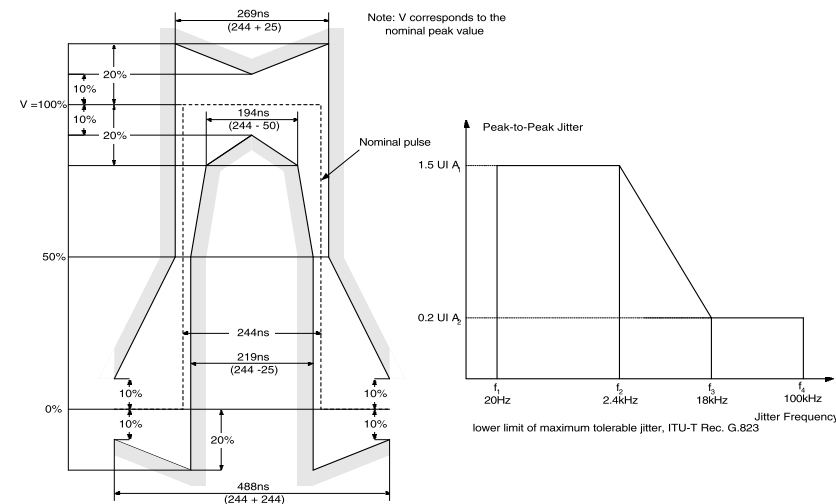
- Recurso introduzido posteriormente que aumenta a robustez do sincronismo por FAS/NFAS.

	Sub-multiframe (SMF)	Frame number	Bits 1 to 8 of the frame							
			1	2	3	4	5	6	7	8
Multiframe	I	0	C_1	0	0	1	1	0	1	1
		1	0	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		2	C_2	0	0	1	1	0	1	1
		3	0	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		4	C_3	0	0	1	1	0	1	1
		5	1	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		6	C_4	0	0	1	1	0	1	1
	II	7	0	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		8	C_1	0	0	1	1	0	1	1
		9	1	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		10	C_2	0	0	1	1	0	1	1
		11	1	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		12	C_3	0	0	1	1	0	1	1
		13	E	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}
		14	C_4	0	0	1	1	0	1	1
		15	E	1	A	S_{a4}	S_{a5}	S_{a6}	S_{a7}	S_{a8}

- C_1 a C_4 carregam o CRC-4, calculado com $G(x) = x^4 + x + 1$, da SMF anterior.
- Não há encadeamento: ao calcular o CRC-4 de uma SMF, os campos C_1 a C_4 dessa SMF entram zerados.
- Agrupar duas SMFs dá espaço para a sequência de alinhamento e para os bits E no bit 1 dos NFAS.

Interface física E1 segundo ITU-T G.703

- Par simétrico de $120\ \Omega$ ou coaxial de $75\ \Omega$
- Pulso nominal retangular, largura 244 ns
- Máscara de pulso e jitter máximo padronizados
- *O padrão não define só os bits; define a forma de onda no conector.*



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

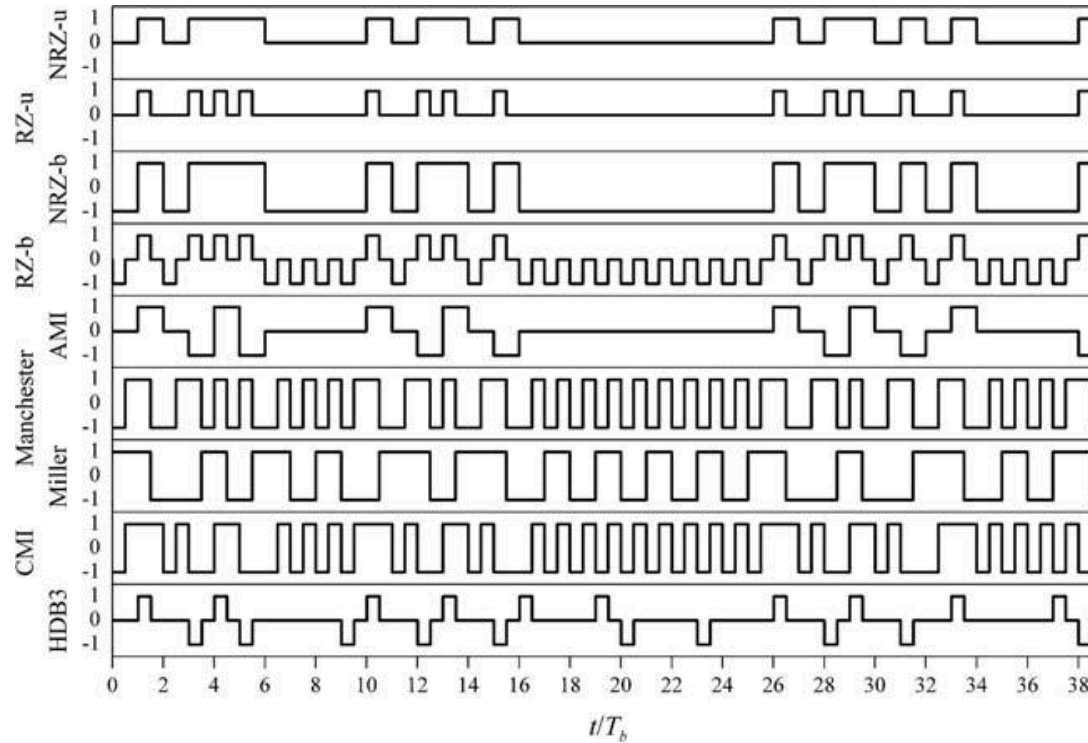
Requisitos de um bom código de linha

Código de linha = regra que transforma bits em forma de onda elétrica no cabo.

1. Garantir transições suficientes para recuperação de clock
2. Evitar componente DC e *baseline wander*
3. Usar banda de forma eficiente
4. Ter complexidade aceitável
5. Se possível, permitir detecção de certas falhas

Qual combinação desses objetivos levou a indústria do E1 ao HDB3?

Panorama visual dos códigos de linha



Guimarães, *Digital Transmission*, 2009, obtido de doi.org.

Mesma sequência binária, formas de onda diferentes, compromissos diferentes.

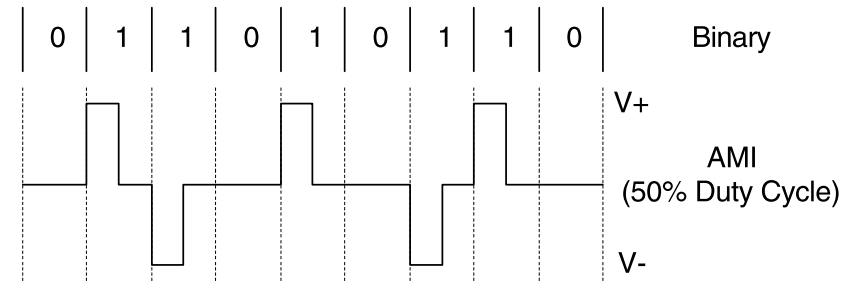
NRZ e RZ: os pontos de partida

- **NRZ unipolar:** simples, mas tem DC e pode ficar “parado” por muito tempo.
- **NRZ bipolar (antipodal):** sem DC para bits equiprováveis, mas pode ter poucas transições.
- **RZ:** volta a zero dentro do intervalo de bit.
 - Vantagem: mais transições.
 - Desvantagem: ocupa mais largura de banda.

Bons códigos didáticos, mas não resolvem bem o problema prático do E1.

AMI: o ancestral direto do HDB3

- Bit 1 → pulso alternando polaridade
- Bit 0 → ausência de pulso
- Vantagens: sem DC, capacidade de detectar violação.
- **Problema fatal para E1:** longas sequências de 0 não carregam relógio.



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

HDB3 = “AMI + remédio para sequências longas de zero”.

Manchester, Miller e CMI: comparação breve

Código	Características	Uso típico
Manchester	Sempre há transição no meio do bit; robusto para clock recovery; ocupa mais banda	Ethernet 10 Mbps (voltará no módulo 3)
Miller	Codificação por transições; Modified Miller usado em RFID (ISO/IEC 14443 Type A)	Cartões sem contato
CMI	Sem DC; boa atividade de transições; padronizado na G.703 para taxas mais altas do PDH	Interfaces PDH de taxa mais alta

HDB3: a ideia em uma frase

AMI com substituição controlada de 4 zeros consecutivos

- Quando o fluxo ficaria sem transições por tempo demais, o codificador injeta pulsos especiais.
- Esses pulsos preservam a média DC próxima de zero e criam um padrão detectável no receptor.

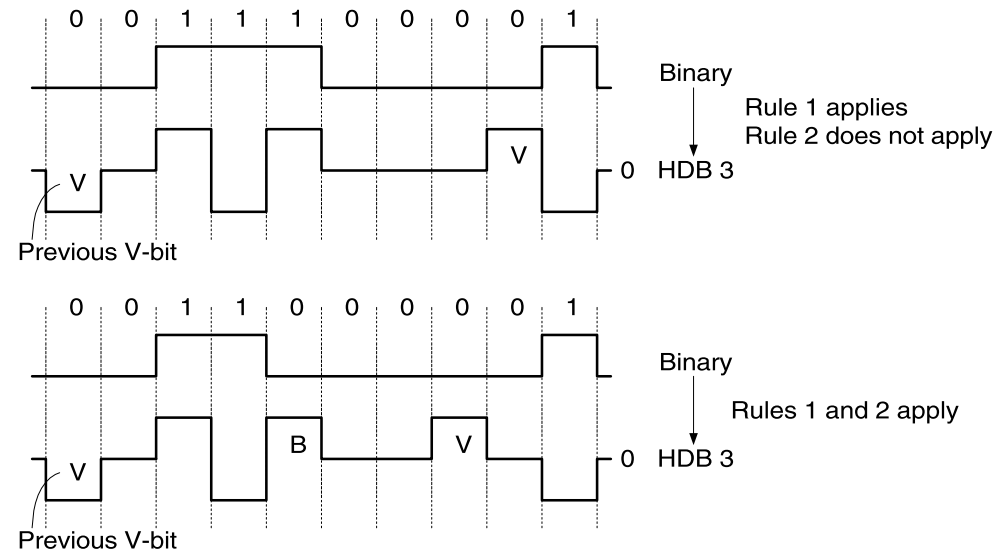
Esse é o compromisso exato de que o E1 precisava.

HDB3: regras 000V e B00V

- Regra 1: 0000 → 000V
- Regra 2: se o número de pulsos desde a última violação for par, usar B00V
- B = pulso bipolar “normal” (obedece à alternância AMI)
- V = pulso de **violação** (mesma polaridade do pulso anterior, quebrando a alternância)

B corrige a paridade; V cria a assinatura detectável da substituição.

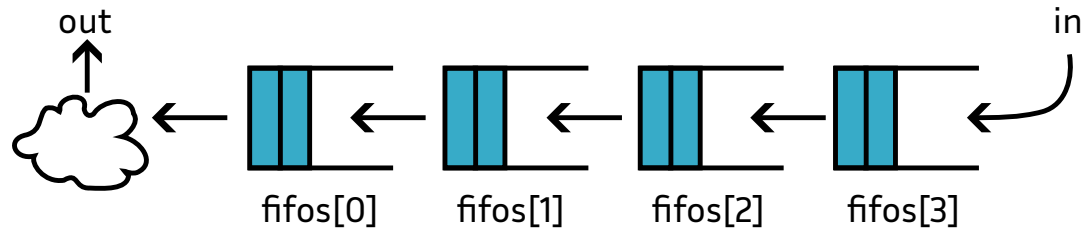
HDB3 em exemplos concretos



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

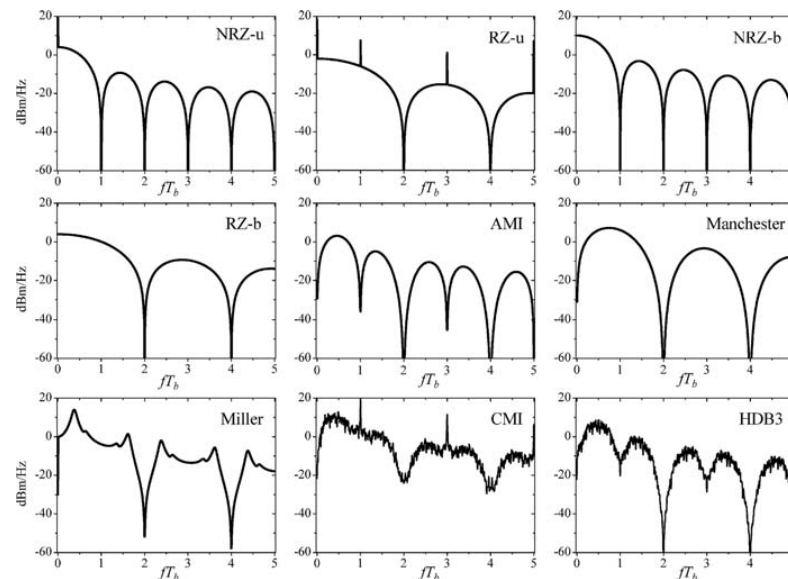
- **Caso 1:** basta 000V.
- **Caso 2:** precisa B00V para manter ausência de DC; as violações aparecem **só** onde havia 4 zeros consecutivos.

Como o receptor desfaz HDB3



- O decodificador detecta padrões impossíveis em AMI puro.
- Se aparecer violação compatível com 000V ou B00V, os 4 símbolos viram 0000.
- A implementação usa **4 FIFOs** de 1 elemento para ter *look-ahead* local.
- Conecta diretamente a teoria do código de linha à arquitetura de hardware.

Densidade espectral dos códigos de linha



Guimarães, *Digital Transmission*, 2009, obtido de doi.org.

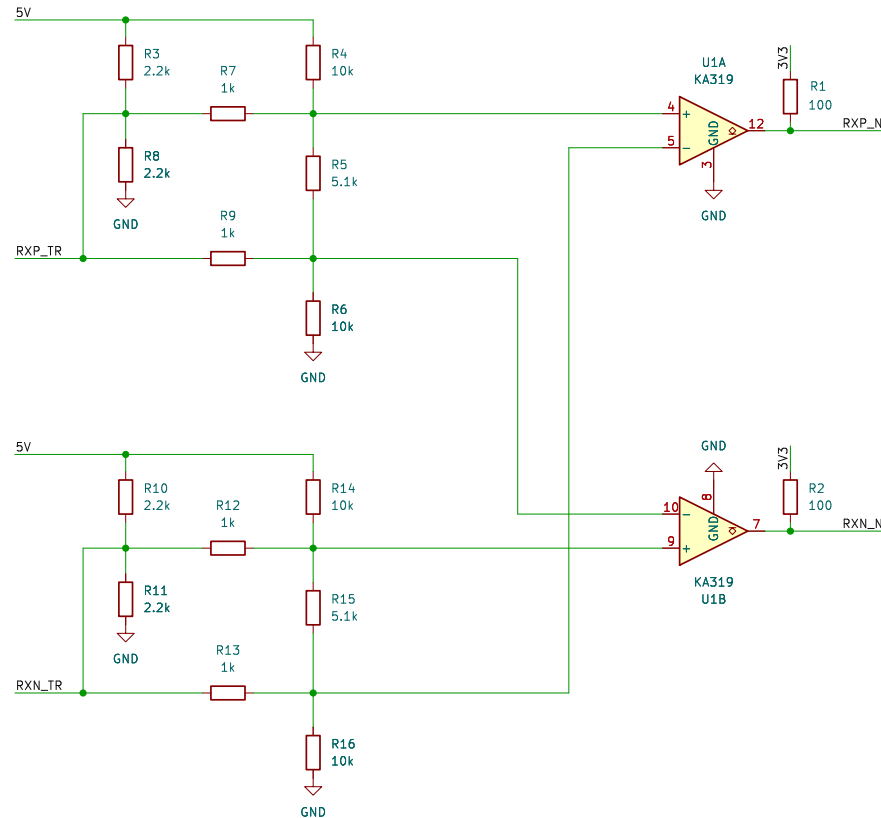
- NRZ unipolar concentra energia em DC. AMI e HDB3 têm nulo em DC.
- HDB3 **preserva** as vantagens espectrais do AMI e corrige o problema de sequências de zero.

Por que o E1 escolheu HDB3

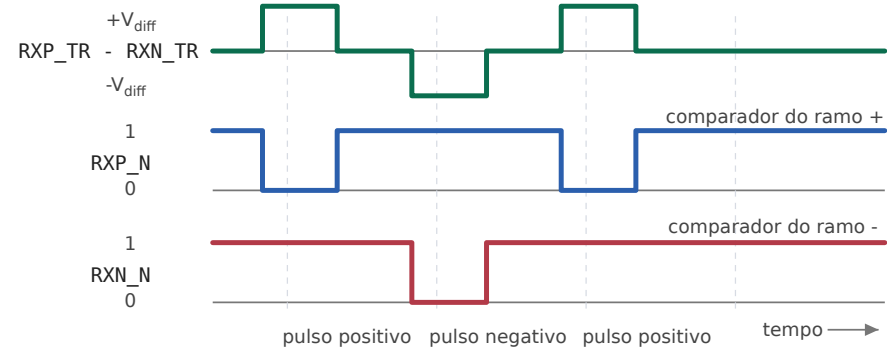
- Sem DC → compatível com acoplamento por transformador.
- Máximo de 3 zeros consecutivos → recuperação de clock viável.
- Violação controlada → ajuda a detectar padrões inválidos.
- Eficiência espectral melhor que Manchester.
- Complexidade bem menor que soluções com DSP pesado.

HDB3 é uma escolha coerente para enlaces PCM a 2,048 Mbit/s em par metálico balanceado.

Hardware da prática: recepção e transmissão E1



Entrada diferencial e saídas do front-end RX



- **RX:** dois comparadores detectam pulsos positivos e negativos.
- **TX:** os próprios pinos do FPGA forçam corrente em um sentido ou no outro.
- Placa pensada para cabos curtos de bancada, não para enlaces longos.

Receptor de laboratório vs. receptor comercial

Na nossa prática:

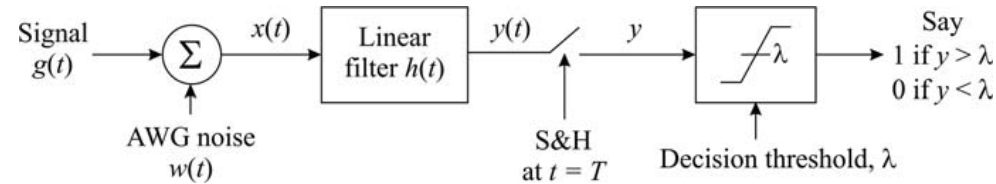
- Transformador + comparadores
- Limiar fixo
- Oversampling + DPLL digital

Em LIUs comerciais de E1:

- Equalização / compensação de linha
- Modos *short-haul* / *long-haul*
- *Slicer* / decisão
- *Clock-data recovery*
- *Jitter attenuator*
- Decodificação de linha

O mercado frequentemente esconde no front-end parte do que nós estamos explicitando didaticamente.

Detectar um pulso recebido: problema de decisão



Guimarães, *Digital Transmission*, 2009, obtido de doi.org.

- Retomando o módulo 1: o receptor ideal quer maximizar a separação entre “pulso presente” e “pulso ausente”.
- O formalismo clássico leva a filtro casado / correlator.
- No E1 de bancada, o front-end já nos entrega decisões ternárias “digitalizadas”.

Por que na Prática 2 um limiar simples basta

- Cabos curtos, atenuação pequena, forma de onda limpa.
- Transformador + comparadores comprimem a decisão em três estados: **P, Z, N.**
- O gargalo didático deixa de ser “como projetar o melhor filtro” e passa a ser:
 - Amostrar no instante certo
 - Reconstruir o clock
 - Entender o código de linha

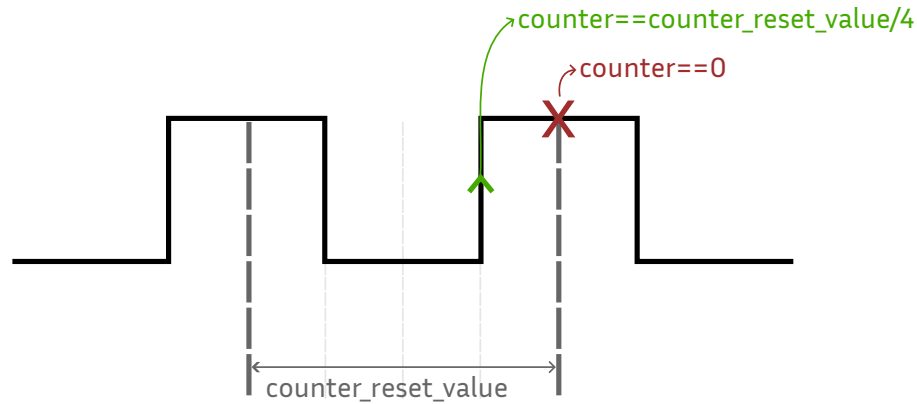
Por isso a prática concentra esforço em DPLL e HDB3, não em equalização adaptativa.

Recuperação de temporização: o problema central

- O transmissor e o receptor têm relógios próprios.
- Mesmo um erro pequeno de frequência, acumulado ao longo do tempo, desloca o instante ótimo de amostragem.
- Se o receptor amostrar cedo ou tarde demais:
 - a margem contra ruído cai;
 - o HDB3 pode ser interpretado errado.

Como usar as transições do sinal recebido para corrigir continuamente nosso relógio local?

DPLL da Prática 2: funcionamento



- Contador roda no clock sobreamostrado.
- Quando chega a zero → amostragem do símbolo.
- Ao detectar o início de um pulso, verifica se a borda chegou cedo, no tempo ou tarde.
- Ajuste aplicado ao próximo período:
 - $\text{max} - 1$ (borda cedo)
 - max (no tempo)
 - $\text{max} + 1$ (borda tarde)

Interpretação em blocos: detector de fase + NCO

- A DPLL implementa um laço de realimentação negativa.
- Entrada: fase observada das bordas do sinal.
- Referência interna: fase esperada da borda.
- Erro de fase $\rightarrow$ comando discreto $\{-1, 0, +1\}$.
- Comando $\rightarrow$ ajuste no período do contador.

$$\text{período}_{k+1} = \text{período}_{\text{nominal}} + u_k, \quad u_k \in \{-1, 0, +1\}$$

Não é magia: é uma malha de controle digital extremamente simples.

Por que o laço converge

- Borda chega cedo → encurtamos o próximo período.
- Borda chega tarde → alongamos o próximo período.
- Correção sempre no sentido **oposto** ao erro → **realimentação negativa**.

Modelo discreto útil:

$$e_{k+1} \approx e_k + K\varepsilon - u_k$$

- e_k : erro de fase acumulado
- ε : erro fracionário entre os clocks
- K : intervalos de bit até a próxima transição útil
- Em regime travado, o erro oscila dentro de uma faixa pequena imposta pela quantização do oversampling.

Oversampling: o que ele proporciona

- Mais oversampling = maior resolução temporal.
- Maior resolução = detector de fase menos grosseiro.
- Menor erro residual de quantização.
- Mais margem para jitter e pequenas diferenças entre clocks.
- Mínimo conceitual: 4 amostras por bit.
- Na prática, **16×** ou **32×** dão um laço muito mais estável.

O mínimo teórico e o mínimo confortável não são a mesma coisa.

Limites teóricos desta DPLL em HDB3

- A DPLL só corrige quando há transição observável.
- HDB3 garante no máximo **3 zeros consecutivos**.
- Na pior hipótese: máximo de **4 intervalos de bit** sem oportunidade de correção.

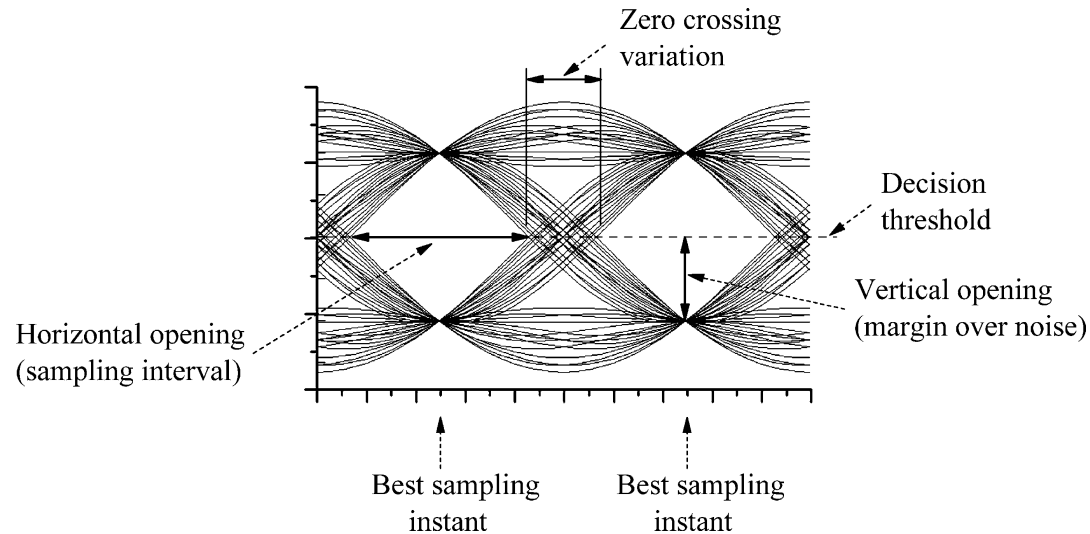
$$|\varepsilon|_{\max} \approx \frac{1}{K \cdot M}$$

- $K \leq 4$: pior espaçamento entre transições
- M : fator de oversampling

Para $M = 32$: $|\varepsilon|_{\max} \approx \frac{1}{128} \approx 0,78\%$

Código de linha e clock recovery não podem ser ensinados separadamente.

Diagrama de olho aplicado ao E1



Guimarães, *Digital Transmission*, 2009, obtido de doi.org.

- Instante ótimo de amostragem = máxima abertura vertical do olho.
- Fechamento vertical: ruído e ISI. Fechamento horizontal: jitter.
- A DPLL mantém a amostragem no centro da abertura.

Comutação por circuitos vs. comutação por pacotes

- Telefonia clássica: rede comutada por **circuitos**.
- IP: rede comutada por **pacotes**.
- O E1 foi concebido no mundo da comutação por circuitos.
- Na Prática 2, colocamos quadros HDLC com tráfego IP dentro de um enlace E1.

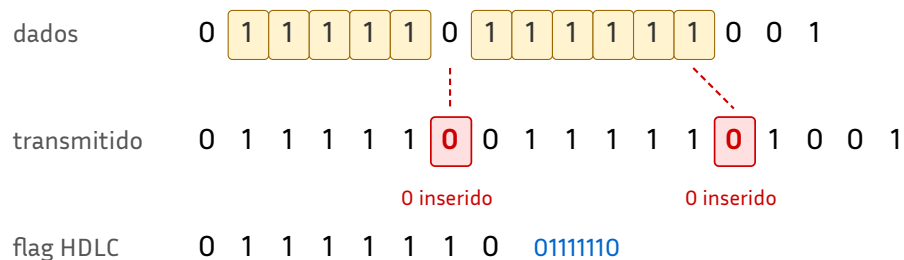
A prática é uma instância concreta de “pacotes sobre infraestrutura historicamente pensada para circuitos”.

O dual também existe: redes de pacotes podem emular circuitos virtuais.

HDLC: papel na cadeia

- Depois de recuperar bits e timeslots, ainda falta **delimitar quadros**.
- Esse é o papel do HDLC.
- No nosso pipeline:
 - E1Unframer descobre o canal.
 - HDLCUnframer descobre o pacote.
- Sem HDLC, teríamos apenas um fluxo síncrono contínuo de bits, sem fronteiras.
- **Flag HDLC:** 01111110; em ocioso, transmite-se uma sequência contínua de flags.
- Um quadro começa depois de uma flag e termina antes da próxima.

Bit stuffing: transparência orientada a bit



Regra: após cinco 1s consecutivos nos dados, inserir um 0.

- Após cinco 1s consecutivos no payload, **inserir um 0**.
- No receptor: se vier 111110, descartar o 0.
- Resultado: a sequência de flag **nunca** aparece acidentalmente dentro do conteúdo.

Da flag ao ping

- Após o desembrulho HDLC, o payload é um quadro **cHDLC** contendo um datagrama IPv4 com ICMP Echo.
- O ICMPReplier é um datapath mínimo, capaz de:
 - reconhecer ICMP Echo Request
 - trocar endereços
 - recalcular checksums
 - anexar FCS (Frame Check Sequence)
- Teste concreto: **ping no Cisco → resposta na FPGA.**

Detecção de erros: por que o CRC

- Quadros podem ser corrompidos na linha.
- Paridade simples não basta — queremos detecção com probabilidade altíssima e custo mínimo em hardware.
- Esse é exatamente o nicho do **CRC**.

Mensagem como polinômio sobre GF(2)

- Em vez de pensar a mensagem como um inteiro, pensamos como um polinômio em x .
- Exemplo: 01001000 $\rightarrow x^6 + x^3$.
- Cada bit 1 vira um termo. Grande vantagem: **não há carry entre posições**.

GF(2): soma e subtração viram XOR

No CRC trabalhamos em GF(2): coeficientes são 0 ou 1.

$$1 + 1 = 0 \quad 1 - 1 = 0 \quad 1 + 0 = 1$$

- Subtrair um polinômio equivale a somar, que equivale a **XOR** termo a termo.

É isso que torna a divisão polinomial implementável com portas XOR e registradores.

Divisão polinomial: passo a passo

$$M(x) = x^6 + x^4 + x^3, G(x) = x^4 + x + 1$$

Dividendo: $M(x) \cdot x^4 = 1011000000$, Divisor: $G(x) = 10011$

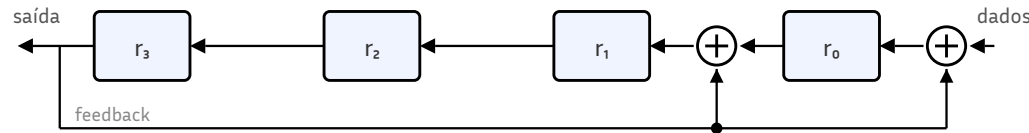
10011	1	0	1	1	0	0	0	0	0	0	0
	1	0	0	1	1						
	0	0	1	0	1	0	0	0	0	0	0
		1	0	0	1	1					
		0	0	1	1	1	0	0	0	0	
			1	0	0	1	1				
			0	1	1	1	1	0	0		
				1	0	0	1	1			
				0	1	1	0	1	0		
					1	0	0	1	1		
					0						
					0	1	0	0	1		

← resto (CRC)

- Transmitir = anexar o resto de $M(x) \cdot x^r \div G(x)$.
- Receber = dividir o quadro inteiro por $G(x)$.
- **Resto zero → quadro consistente.**

Por que CRC é amigável para FPGA

$$G(x) = x^4 + x + 1$$



- A divisão polinomial em $GF(2)$ pode ser implementada como **LFSR**.
- Cada bit de entrada avança o registrador.
- Os taps do polinômio viram XORs.
- Sem multiplicadores, sem divisores gerais.

Capacidade de detecção do CRC

- Todo CRC **bem escolhido** detecta todos os erros de 1 bit.
- Detecta todos os *bursts* com comprimento $\leq$ grau do polinômio.
- Para erros mais longos, importam os fatores do polinômio.
- Na nossa prática (HDLC): **CRC-16-CCITT** ($x^{16} + x^{12} + x^5 + 1$).
- CRC-32 aparecerá no módulo 3, ao estudar Ethernet.

Se o gerador tem grau r , um burst de comprimento $L \leq r$ não pode ser múltiplo exato de $G(x)$, portanto é sempre detectado.

Mapa do quadro E1: voz, sinalização e dados

- **TS0** → sincronismo, alarmes, eventualmente CRC-4.
- **TS16** → sinalização CAS (ABCD).
- **TS1–15 e TS17–31** → 30 canais de 64 kbit/s.

Na telefonia clássica, convém separar três planos:

- **Sinalização de assinante/acústica** → telefone ↔ central
- **Sinalização de linha** → supervisão entre terminações do tronco (vai em TS16 no E1/CAS)
- **Sinalização de registro** → dígitos, categoria e controle (pode ir in-band, como no R2/MFC)

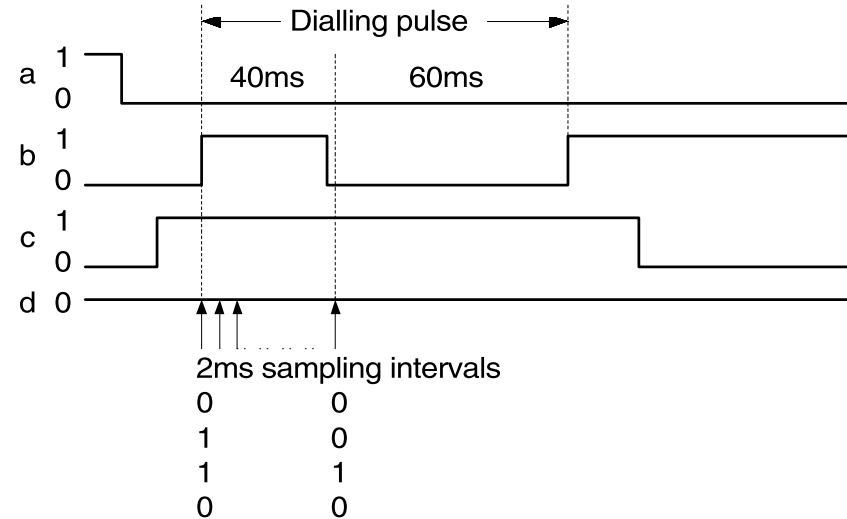
Num assinante analógico, o telefone não escreve TS16 diretamente — a central converte o estado local nos bits ABCD. Num tronco E1 empresarial, o PABX gera/interpreta os bits ABCD.

E&M: sinalização de linha para troncos

- E&M é uma família de sinalizações de linha para troncos telefônicos.
- Função: supervisionar o tronco — **livre, ocupação, atendimento, desligamento**.
- E&M **não define**, por si só, como os dígitos são enviados.

Exemplo Cisco E&M digital:

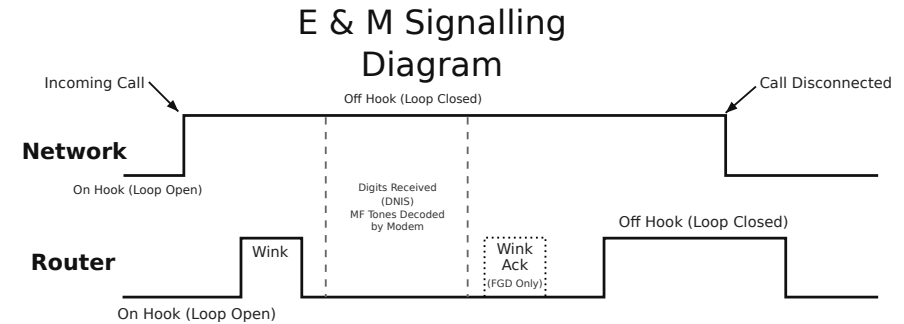
- *idle/on-hook* = ABCD 0000
- *seized/off-hook* = ABCD 1111



Tibbs, *Pocket Guide to The world of E1*, 2002, obtido de web.fe.up.pt.

E&M: caso típico de tie-line entre PABXs

1. Tronco em *idle*.
2. Origem faz *seizure* do tronco.
3. Em variantes *wink*, o destino devolve pulso curto = “estou pronto”.
4. Caminho de áudio é aberto; usuário ouve tom de discar remoto.
5. Usuário envia dígitos do destino (DTMF pelo tronco tomado).
6. Equipamento remoto roteia a chamada.
7. Destino atende → conversa pelo tronco.
8. Desligamento → tronco volta a *idle*.



Cisco, *Understanding How Digital T1 CAS Works in IOS Gateways*, 2007, obtido de cisco.com.

A supervisão é simples; o restante da chamada acontece do lado remoto após a abertura do caminho de áudio.

R2/MFC: sinalização em dois planos no mesmo E1

- Na sinalização R2, há **dois planos**:
 - **Sinalização de linha** (*line signaling*) → bits ABCD em TS16
 - **Sinalização de registradores** (*inter-register signaling*) → tons multifrequenciais no próprio canal de voz
- R2/MFC foi a sinalização predominante em troncos digitais E1 no Brasil.
- MFC é **compelida**: cada sinal para a frente espera um sinal de volta antes do próximo passo.

No R2/MFC, o mesmo timeslot que depois carregará PCM de voz é temporariamente usado para sinalização de registradores.

R2 digital: bits A/B por fase da chamada

Fase	Orig→Dest (af bf)	Dest→Orig (ab bb)	Interpretação
Tronco livre	1 0	1 0	<i>idle</i>
Ocupação	0 0	1 0	origem toma o circuito
Em progresso	0 0	1 1	destino confirmou; MFC pode começar
Atendimento	0 0	0 1	chamada atendida
Conversação	0 0	0 1	canal carrega PCM de voz
Deslig. p/ trás	0 0	1 1	destino iniciou liberação
Deslig. p/ frente	1 0	X 1	origem iniciou liberação
Confirmação	1 0	1 0	tronco volta a <i>idle</i>

No R2 digital internacional (Q.421), a supervisão usa só A e B; C fica fixo em 0 e D em 1.

R2/MFC: diálogo compelido entre registradores

MFC não é “mandar dígitos” — é um **diálogo compelido**.

Exemplo compacto:

1. Origem envia sinal *forward* (dígito).
 2. Destino responde A1 = “envie o próximo dígito”.
 3. Após alguns dígitos, destino responde A3 = “mude para outro grupo”.
 4. Origem envia G2 = “categoria do chamador”.
 5. Destino responde B1 = “assinante livre com tarifação”.
- Sinais *forward* = frequências altas; sinais *backward* = frequências baixas.
 - Durante a fase MFC, o tronco está ligado ao bloco de registradores, **não ao usuário**.

DTMF, pulso decádico e MFC: quem é in-band?

Mecanismo	Natureza	No E1
DTMF	Tons na faixa de voz	Pode atravessar a banda de áudio do tronco
Pulso decádico	Abertura/fechamento do loop no acesso analógico	Convertido em CAS/ABCD no TS16 pelo gateway
MFC / MFC-R2	Tons multifrequenciais na faixa de voz	In-band no canal de tráfego entre centrais

Nem todo mecanismo de discagem usa o canal de voz: DTMF e MFC usam banda de voz; pulso decádico, ao ser transportado por E1/CAS, vira sinalização no TS16.

Da telefonia CAS ao SIP: panorama evolutivo

Época	Tecnologia	Onde anda a sinalização
1960s	E&M / troncos analógicos	Corrente e tensão nos fios
1970s	E1 CAS / R2-MFC	TS16 + tons no canal de voz
1980s	SS7 / TUP / ISUP	Canal comum separado (CCS)
2000s	SIP / VoIP	Mensagens em rede de pacotes

Analogia: *seizure* INVITE, *ringing* 180 Ringing, *answer* 200 OK, *clear* BYE.

Este slide é panorâmico; os detalhes de VoIP ficam fora deste módulo.

Conexão com equipamentos de mercado

- Gateways e roteadores com interfaces E1 permitem fracionar timeslots entre voz e dados.
- O conceito importante: o equipamento real precisa:
 - terminar a física E1
 - interpretar *line code* e *frame*
 - fracionar timeslots
 - encaminhar voz e/ou dados
- Padrões CAS/ABCD de *idle*, *seize* etc. são configuráveis por equipamento.

Isso reforça por que vale separar a lógica da sinalização do mapeamento específico de bits.

BSV: o mínimo para ler a prática

- BSV = Bluespec SystemVerilog (HDL com abstrações de alto nível).
- Três conceitos-chave:
 - **Módulos:** unidades de hardware com estado.
 - **Interfaces:** portas tipadas (Put/Get).
 - **Regras:** ações automáticas quando a guarda está satisfeita.

```
interface HDB3Decoder;  
  interface Put#(Symbol) in;  
  interface Get#(Bit#(1)) out;  
endinterface
```


BSV: regras, FIFOs e composição

- rule: ação automática quando a guarda está satisfeita.
- FIFOF: forma comum de desacoplar estágios.
- mkConnection: conecta interfaces sem cola manual.

```
rule select_desired_ts;  
  match { .ts, .value } <- unfr.out.get;  
  if (ts != 0) unhdlc.in.put(value);  
endrule
```

```
mkConnection(hdb3enc.out, tlio.in);  
mkConnection(tlio.out, hdb3dec.in);
```

A arquitetura da prática é essencialmente “encanamento síncrono” entre blocos.

Mapa mental para entrar na prática

- Física do enlace: G.703, pulso, HDB3, comparadores.
- Temporização: oversampling + DPLL.
- Estrutura E1: G.704, quadros, TS0, TS16, sincronismo.
- Enlace de dados: HDLC + bit stuffing.
- Detecção de erros: CRC.
- Telefonia clássica: voz PCM + sinalização CAS / E&M / R2.
- Implementação: blocos em BSV conectados por FIFOs e regras.

Se o aluno entende esse mapa, a Prática 2 deixa de ser “um monte de arquivos” e vira um sistema de comunicação coerente.